

Tokenomics Production Runtime

Named production tokens, Token Bank assets, quantitative flows, adapters, and model-token cost governance

1. Production Objective

The production goal is to turn Tokenomics from a research doctrine into a downloadable runtime that anyone can attach to an AI system. The kernel should work above GPT, cloud APIs, local models, native AI systems, or human-in-the-loop workspace agents. It should not depend on one model provider. Its job is to govern cognition mathematically before and after generation.

`Classify -> allocate -> execute -> score -> consolidate`

The model still generates, reasons, calls tools, or produces artifacts. Tokenomics names the internal tokens, applies their formulas, quantifies salience and cost, decides how deep the task should go, what modules should activate, how output should be scored, and what should become reusable memory.

2. Market Pressure: Paid Token Exposure

The production urgency is rising because AI coding and agent platforms are moving toward usage-based billing. When input, output, cached context, model choice, tool calls, and long-running agents all affect cost, users need a formula kernel before the paid model call. Tokenomics should therefore operate as cost governance, not merely after-action scoring.

The core economic move is local cognition first, external model spend second. Stable formulas and approved banked cognitive assets should handle salience, budget, routing, cost, risk, compression, verification, memory, drift, and arbitration locally. Paid model tokens should be reserved for unresolved reasoning, fresh generation, verification, and tool work.

3. System Boundary

Layer	Responsibility
User / app	Submits the task and receives an answer, artifact, or decision.
Tokenomics runtime	Names tokens, retrieves banked tokens, applies formulas, allocates, routes, scores, logs, and extracts reusable rules.
Model adapter	Connects the runtime to GPT, a cloud model, local model, native AI, or human operator.

Layer	Responsibility
Execution engine	Generates text, calls tools, edits files, creates documents, or performs domain work.
Benchmark/memory store	Records task results and stores reusable rules for future cost reduction.

4. Production Token Ontology

Production Tokenomics uses named tokens. Each token carries a formula role, quantitative output field, flow stage, and required inputs. These are system primitives, not examples.

Token	Formula role
DECISION_TOKEN	$D = f(\text{correctness}, \text{consequence}, \text{option_elimination})$
ACTION_TOKEN	$A = f(\text{next_step_clarity}, \text{executability}, \text{artifact_readiness})$
RISK_TOKEN	$R = \text{exposure} * \text{probability} * \text{mitigation_coverage}$
COMPRESSION_TOKEN	$C = (\text{information_retained} + \text{action_clarity} + \text{risk_preserved}) / \text{output_tokens}$
MEMORY_TOKEN	$M = \text{reuse_frequency} * \text{future_cost_avoidance} * \text{confidence}$
SALIENCE_TOKEN	$S_i = U_i + R_i + M_i + T_i + N_i - K_i$
BUDGET_TOKEN	$B_i = B_{\text{total}} * S_i / \text{sum}(S)$
ROUTING_TOKEN	$\text{Route}_j = \text{argmax}(E[\text{value}_j] - E[\text{cost}_j] - E[\text{latency}_j])$
COST_TOKEN	$\text{Cost} = (\text{in} * r_{\text{in}} + \text{out} * r_{\text{out}} + \text{cache} * r_{\text{cache}}) / 1e6$
DRIFT_TOKEN	$\text{Drift} = (\text{score_prev} - \text{score_now}) + (\text{waste_now} - \text{waste_prev})$

5. Production Token Bank

The Token Bank is the runtime's local store of approved reusable cognition. It is not a sample prompt library. A banked token is a versioned cognitive asset bound to ontology tokens, task classes, tags, quantitative values, approval status, and estimated local token value. It lets the runtime retrieve already-filled formulas and policies before paying an external model.

Banked token family	Production function
Saliency-to-budget allocation	Preloads the ranking and proportional allocation rule so repeated tasks do not resend allocation doctrine.
Token value and cognitive return	Preloads TV and CRPT scoring logic for post-run evaluation.
Compression fidelity gate	Prevents short outputs from hiding uncertainty or execution constraints.
Paid-token spend governance	Preloads provider cost, routing, privacy, and latency policy before model use.
Approved memory consolidation	Requires approval, versioning, and rollback before durable memory changes.
Production controller flow	Preloads plan -> execute -> usage -> score -> store -> consolidate behavior.
Workspace benchmark families	Preloads INV, EST, CASH, PROP, and AI-ENG scoring focus.

```
BankScore_b = TaskClassMatch_b + OntologyMatch_b + TagMatch_b + TextMatch_b
LocalOffset = FormulaTokenOffset + sum(BankedTokenEstimate_b)
```

6. Portable Runtime Components

Component	Production behavior
Task Classifier	Maps requests to invoice, estimate, proposal, cashflow, architecture, research, code/workspace, strategy, or general.
Production Token Registry	Defines named tokens, formulas, stages, required inputs, and output fields.
Formula Engine	Computes token value, saliency, budget

Component	Production behavior
	allocation, compression efficiency, cost, gain, and drift.
Spend Governor	Estimates external token exposure and warns when cost or token limits are exceeded.
Token Governor	Sets depth and token budget by task class, risk, urgency, and mission relevance.
Salience Engine	Scores decision, risk, artifact, reuse, verification, and implementation targets.
Sparse Router	Selects which engines, tools, or agents should be active for the task.
Output Contract	Defines what must be returned so the model does not ramble or omit key deliverables.
Cognitive Return Scorer	Scores decision quality, actionability, risk control, reuse, learning, accuracy, and waste.
Benchmark Log	Stores JSONL task records for analysis and longitudinal improvement.
Reuse Extractor	Identifies memory/rule candidates that reduce future token cost and error risk.

7. Adapter Strategy

The runtime should be model-agnostic. Adapters should translate a RuntimePlan into whatever the target system needs: system instructions for GPT, API payloads for cloud models, local prompts for native models, or checklist instructions for a human operator.

Target	Adapter behavior
GPT / ChatGPT	Convert plan into system/developer instructions, output contract, and post-response scoring.
Cloud API	Wrap request payloads with task class, budget, salience, and output-contract metadata.
Local model	Generate a compact controller prompt and external benchmark record.

Target	Adapter behavior
Native AI	Expose runtime plan as engine-routing metadata and memory-consolidation hints.
Human workspace	Use the plan as an execution checklist and benchmark rubric.

plan -> execute -> usage -> score

A production adapter must return output text, artifact reference, provider, model, model version, input tokens, output tokens, cached tokens, and metadata. The runtime then scores the result and stores the benchmark record with comparable usage fields.

8. Data Contracts

The production kernel uses RuntimePlan plus contracts for cognitive vectors, production tokens, spend estimates, scorecards, and benchmark records.

Contract	Fields
TaskRequest	task, details, requested_artifact, constraints, known_context, signals, metadata
CognitiveVector	decision_quality, actionability, risk_reduction, compression_gain, memory_reuse, learning_gain, accuracy, waste, latency_cost, privacy_risk, external_cost
ProductionToken	name, formula, vector, applies_to, flow_stage, required_inputs, metadata
BankedToken	id, title, content, token_names, task_classes, tags, quantitative_values, version, status, source, token_estimate
RuntimePlan	task_class, depth, token_budget, budget_allocation, modules, output_contract, salience, risk_notes, reuse_candidates, production_tokens, banked_tokens, flow_plan, external_token_budget, estimated_external_cost, spend_warnings
SpendEstimate	external_input_tokens, external_output_tokens, cached_tokens, estimated_cost_usd, warnings
ScoreCard	decision_quality, actionability, risk_control, reuse_value, learning_gain, accuracy, waste,

Contract	Fields
	total_tokens, crpt
BenchmarkRecord	task, task_class, artifact, provider, model, model_version, usage_tokens, scorecard, failure_mode, consolidated_rule, created_at

9. Current Production Package

The package is `tokenomics-runtime` v1.1.0 with production ontology, Token Bank, formula engine, flow compiler, spend governor, adapters, rubric scoring, SQLite storage, memory approval, rollback, and benchmark suite.

- `tokenomics\_runtime.models`: dataclass contracts for requests, plans, scores, and benchmark records.
- `tokenomics\_runtime.ontology`: named production tokens and formulas.
- `tokenomics\_runtime.token\_bank`: approved reusable cognitive assets tied to ontology tokens.
- `tokenomics\_runtime.formulas`: deterministic formula engine.
- `tokenomics\_runtime.flows`: production flow compiler.
- `tokenomics\_runtime.spend`: external spend governor and model-rate estimates.
- `tokenomics\_runtime.adapters`: adapter protocol plus GPT/cloud, local/native, and planning adapters.
- `tokenomics\_runtime.scoring`: rubric-based scorer with optional human/expert ratings.
- `tokenomics\_runtime.storage`: SQLite benchmark and memory stores.
- `tokenomics\_runtime.benchmark\_suite`: INV, EST, CASH, PROP, and AI-ENG production benchmark families.
- `tokenomics\_runtime.policies`: task classification, depth policy, token budget policy, salience policy, and output contracts.
- `tokenomics\_runtime.runtime`: portable controller that produces plans and score records.
- `tokenomics\_runtime.benchmark`: JSONL benchmark logger.
- `tokenomics\_runtime.cli`: command-line entry point for planning and scoring tasks.
- `tests/test\_runtime.py`: test coverage for classification, bank retrieval, adapters, scoring, storage, memory rollback, and benchmark families.

10. Production Roadmap

Phase	Build target
0. Runtime core	Portable Python controller, CLI, schemas, ontology, formulas, flow compiler, spend governor, tests. Complete.
1. Token Bank	Approved local banked tokens tied to ontology formulas, tags, task classes, and local cost offsets. Complete.
2. Adapter interface	Plan -> execute -> usage -> score with GPT/cloud dry-run and local/native execution. Complete.
3. Scoring and storage	Rubric scoring, optional ratings, JSONL/SQLite benchmark storage, provider/model/version metadata. Complete.
4. Memory backend	SQLite approval, versioning, rejection, rollback, and approved-rule retrieval. Complete.
5. Benchmark suite	INV, EST, CASH, PROP, and AI-ENG production families. Complete.
6. Deployment hardening	Auth, privacy modes, packaging release, docs site, dashboard, provider-specific exact usage adapters.

11. Red-Team Fixes Built

- Real adapter interface added: plan -> execute -> usage -> score.
- GPT/cloud adapter and local/native adapter added.
- Keyword scoring replaced with rubric scoring and optional human/expert ratings.
- SQLite benchmark storage added with provider, model, version, task, and usage metadata.
- Memory consolidation now supports approval, versioning, rejection, and rollback.
- Production benchmark suite added for INV, EST, CASH, PROP, and AI-ENG.

12. Remaining Deployment Risks

- Exact token counts still depend on provider usage fields when live cloud calls are made.

- Rubric scoring becomes stronger with expert ratings or evaluator models in high-stakes benchmarks.
- Provider-specific adapters should be hardened against API differences, retries, rate limits, and privacy rules.
- Benchmark dashboards and statistical comparison reports are next-layer product work.
- Security and privacy rules must be explicit before external deployment.

13. Security and Privacy

A production runtime should never assume all tasks can be logged in full. Benchmark records should allow redaction, metadata-only logging, private memory stores, and local-only operation. For commercial work, invoice data, pricing, client names, and contract details may require restricted storage or no external telemetry.

14. Recommended Next Build

- Create a provider matrix for OpenAI-compatible APIs, GitHub Copilot-style usage, local model runtimes, and native Core Brain engines.
- Add a benchmark dashboard that compares CRPT, accuracy, waste, cost, and drift by provider/model/task family.
- Add policy configuration profiles for OIS, research, generic-user mode, and native AI mode.
- Add privacy modes: no-log, metadata-only, redacted benchmark, and full local benchmark.
- Package the project as an installable release with a docs site and versioned policy files.

15. Conclusion

The first production move is not to build another chatbot. It is to build a portable cognitive runtime that any AI can sit behind. Tokenomics becomes the layer that governs what matters, how much depth is earned, what output must be produced, how success is measured, and what the system should learn for next time. That is how the research framework becomes software.